Siva Subramanyam .B.
: 192572089
CODE : CSA0403
NAME : Operating systems.

SCHEDULING IN A REAL-TIME TICKET
G SYSTEM:
Suitable Scheduling Algorithm:
→ The most suitable scheduling algorithm
ority Scheduling because the processes have
t importance levels.
→ Payment transactions are the highest
y, followed by seat availability check &
confirmation.
Justification:
Priority scheduling executes important tasks
reducing the waiting time for critical
ons.
P_1 (payment) is processed first to avoid
nt delays.
P_2 (seat check) is processed next to quickly
seat availability.
P_3 (ticket confirmation) is processed last
it is less critical.
SAMPLE EXECUTION ORDER:

ss	Task	Priority
	Payment Gateway	High

JUSTIFICATION:

EXECUTION ORDER:

$P_1 \rightarrow P_2 \rightarrow P_3$

TIMELINE:

0ms — P_1 — 5ms — P_2 — 13ms — P_3 — 25ms

d) DRAWBACKS:

The main drawback is starvation. If high-priority processes keep arriving, low-priority processes like ticket confirmation may wait for long time. This can be reduced using the aging technique. Priority Scheduling is the best choice for a railway ticket booking system.

2. DEADLOCK IN A BANKING TRANSACTION SERVER:

a) Resource Allocation Graph:

$T_1 \rightarrow B$ ↑ | A $\leftarrow T_1$

$T_2 \rightarrow C$ ↑ | B $\leftarrow T_2$

$T_3 \rightarrow A$ ↑ | C $\leftarrow T_3$

or simply:

* T_1 holds A and requests B
* T_2 holds B and requests C
* T_3 holds C and requests A

This form the cycle:

$T_1 \rightarrow B \rightarrow T_2 \rightarrow C \rightarrow T_3 \rightarrow A \rightarrow T_1$

b) Has a Deadlock Occured?

Yes, a deadlock has occured.

...le holds that is a result ...on.

JUSTIFICATION:

* T_1 is waiting for B, which is held by T_2.
* T_2 is waiting for C, which is held by T_3.
* T_3 is waiting for A, which is held by T_1.

c) Technique to Fix the Issue:

DEADLOCK PREVENTION:

* The System can prevent deadlock by forcing all transaction to acquire resource in same order.

$$A \rightarrow B \rightarrow C$$

* This removes the possibility of circular waiting, so deadlocks cannot occurs.

d) Redesign for Locking Order:

Use a fixed resource locking order for every Transaction:

→ First Lock Account Database (A)
→ Then Lock Customer Info Database (B)
→ Finally Lock Transaction Logs (C)

Since all transaction follow the same sequence, circular waiting is eliminated & future deadlocks are avoided.

3. MOBILE OS APP SWITCHING LAG:

a) CPU remain idle:

Although many apps are running, most background apps are waiting for user input or I/O operations. Since they are not ready to execute, the CPU may become idle even when several apps are active. Frequent context switching also wastes CPU time.

nDOLESS COMMUNI...

resource & waits for ...

b) The 20ms Time Quantum Helping:

The 20ms Time Quantum helping is hurting performance - With 20 active background apps, the CPU performs many context switches, increasing overhead. This causes delays while switching between the apps like Whatsapp & social media.

c) Recommended Scheduling Method:

Use Priority Scheduling with Round Robin

* Give foreground apps → higher priority.
* Give background apps → lower priority.
* Reduce the time quantum to 10 ms for better responsiveness.

This allows active apps to get CPU time quickly while background apps

d) How will it improve Performance:

→ Foreground apps receive CPU time immediately.
→ Reduces app switching delay.
→ Decreases unnecessary waiting time
→ Improves system responsiveness & overall throughput
→ Provides a smoother multitasking experience for users.

4. DEADLOCK RISK IN AN OPERATING SYSTEM UPDATE:

a) Can this sequence lead to a deadlock?

Yes, this sequence can lead to a deadlock.

Each module holds one resource & waits for another resource that is already occupied by another module. As a result, none of the modules can continue its execution.

b) Circular Wait Condition:

The circular waits:

* M_1 holds R_1 & waits for R_2
* M_2 holds R_2 & waits for R_3
* M_3 holds R_3 & waits for R_1

Cycle:

$M_1 \rightarrow R_2 \rightarrow M_2 \rightarrow R_3 \rightarrow M_3 \rightarrow R_1 \rightarrow M_1$

c) Banker's Algorithm to Avoid Deadlock:

* Before allocating a resource, the OS checks whether the allocation keeps the system safe.
* This prevents the system from entering a deadlock.

d) Improving CPU Utilization:

→ M_1 – Copy Update Files
→ M_2 – Install Drivers
→ M_3 – Update Registry
→ M_4 – Validate System Integrity.

Also, the modules should request resources in the same order

$R_1 \rightarrow R_2 \rightarrow R_3$

4. CLOUD SERVER CPU UTILIZATION ANALYSIS:

GIVEN:

$P = 0.8$

$n = 7$

FORMULA => $U = 1 - p^n$

a) Calculate CPU Utilization:

$U = 1 - (0.8)^7$

$U = 1 - 0.2097$

$U \approx 0.79$

b) Interpretation:

A CPU utilization of 79% indicates that the server is efficiently utilized. The CPU is busy most of the time, but there is still some idle time because many VM processes are waiting for network or disk operations.

c) Effect of Changing the number of VMs.

* Increasing the number of VMs.
* Too many VMs
* Reducing the number of VMs.

d) OS - Level Strategies to Improve CPU Utilization:

* Use efficient CPU scheduling to distribute CPU time fairly among VMs.
* Improve I/O performance using faster storage to reduce waiting time.

6. CPU SCHEDULING SCENARIO:

a) Root Cause of the Problem:

During peak usage, many processes compete for CPU at the same time. This causes high CPU contention, increased waiting time, frequent context switching, & reduced system throughput, resulting in poor performance.

b) Suitable OS Technique / Algorithm:

The most suitable scheduling algorithm is Priority Scheduling. It gives higher priority to important processes while allowing lower-priority processes to execute when the CPU is free.

c) Justification:

* Improves utilization CPU
* Reduces response time.
* Increases throughput
* Provides better System.

d) Improved System Configuration:

High → Medium → Low Priority.

EXAMPLE:

P_1 (Critical) → P_2 (Normal) → P_3 (Backgroun

If multiple processes have the same priority, Round Robin can be used among them to ensure fairness.

e) Limitation:

* Starvation, low-priority may wait.

..IICATION :

This problem can be reduced by using the aging technique, which gradually increases.

7. PROCESS MANAGEMENT IN AN E-COMMERCE:

a) Processes Involved :

The System includes the following :

→ Product Search
→ Payment Processing
→ Inventory Management
→ User Authentication.

b) Program :

A Stored set of instructions.
Eg : the payment application

Process :

A program that is currently executing
Eg : Customer's payment transaction.

c) Role of Process Control block (PCB)

→ Process ID (PID)
→ Process State
→ CPU registers
→ Program Counter
→ Memory information
→ Scheduling information.

d) Process States of Payment Transaction :

New - Payment request is created
↓
Ready - Waiting for CPU
↓
Running - Payment is processed
↓

Waiting - Waiting for Bank / server response
↓
Ready - ~~[illegible]~~
Ready - Returns to CPU queue.
↓
Running - Completes transaction
↓
Terminated - Payment finishes

e) Impact of Context Switching:

During festive sales, frequent context switching increases CPU overhead and response time - Too many switches can reduce overall system performance

f) Consequences of Excessive Process Creation:

* High memory usage
* Increased CPU overhead
* More context switching
* Reduced throughput.

g) Efficient Process Hierachy:

Main E-commerce Server
User Authentication.
Product Search
Shopping Chart
Payment Processing
Inventory Management
Shipment Tracking.

h) Need for Multiprogramming:

It allows multiple processes to run simultaneously, improving CPU utilization, reducing

7. INTER-PROCESS ...

a) idle time, & serving many users efficiently.

i) Strategies to improve process management

* Priority Scheduling.
* Load balancing.
* Memory Management.

b)

j) Process Lifecycle Model.

Create → Ready → Running → Waiting
↓
Terminated ← Running ← Ready

8. PROCESS SCHEDULING IN A SMART HOSPITAL:

a) Scheduling Challenges:

* Emerging services need immediate CPU access.
* Delays in accessing patient records.

b) Impact on Emergency Care

Fast scheduling improves patient care ← response time.

c) Algorithm:

Priority Scheduling is the best choice due to emergency services.

d) Starvation & Remedy:

PROBLEM:

Routine tasks may suffer starvation.

SOLUTION:

Use aging, where the priority of waiting processes is gradually increased.

9. INTER-PROCESS COMMUNICATION.

a) Cooperating Processes :-

→ Traffic Signal control
→ CCTV Camera
→ Weather Monitoring
→ Public Transport System.

b) Communication Requirement :

These process must exchange real-time information such as traffic conditions, weather updates.

c) Recommended IPC Mechanism :

Message Passing is the best choice because the smart city :

d) Impact of Communication :

* Delay in traffic signal updates.
* Reduced public safety.
* Increased traffic congestion.

e) Fault - Tolerant IPC :

→ Message Queue
→ backup communication servers.
→ Automatically resend.

10. THREADS & MULTITHREADING MODELS :

1. Threads involved :

* Video Decoding
* Buffering
* Advertisement Insertion

b) Consequences of Thread Blocking.

* Video may freeze
* Buffering delays increase
* Audio & video become unsynchronized

c) System Behaviour with 10 x More Users:

* CPU & memory usage increases
* More Threads are created.
* Response time increases.

d) Improving Thread:

→ Mutex locks
→ Semaphores
→ Avoid race conditions.